

Preface

Test-Driven Development:

- Don't write a line of new code unless you first have a failing automated test.
- Eliminate duplication.

Two simple rules, but they generate complex individual and group behavior. Some of the technical implications are:

- You must design organically, with running code providing feedback between decisions
- You must write your own tests, since you can't wait twenty times a day for someone else to write a test
- Your development environment must provide rapid response to small changes
- Your designs must consist of many highly cohesive, loosely coupled components, just to make testing easy

The two rules imply an order to the tasks of programming:

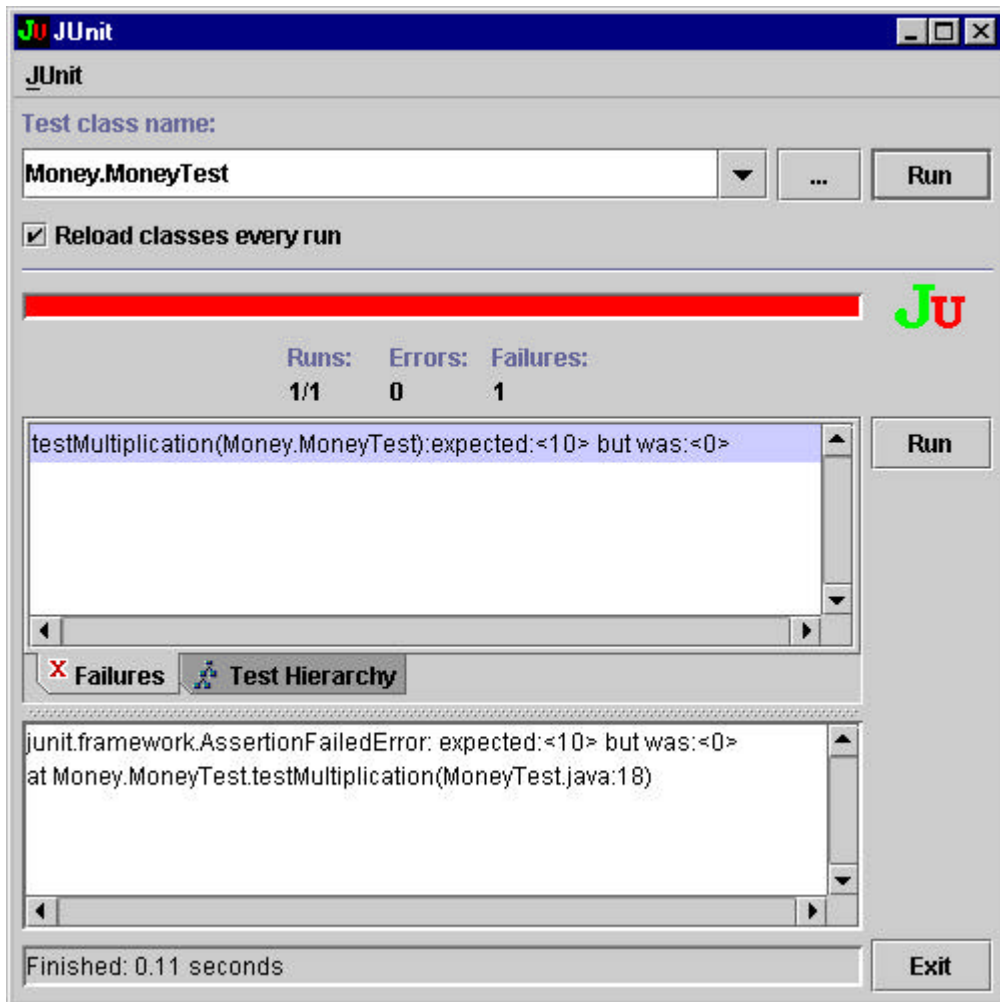
- Red—write a little test that doesn't work, perhaps doesn't even compile at first
- Green—make the test work quickly, committing whatever sins necessary in the process
- Refactor—eliminate all the duplication created in just getting the test to work

Red/green/refactor. The TDDs mantra.

Assuming for the moment that such a style is possible, it might be possible to dramatically reduce the defect density of code and make the subject of work crystal clear to all involved. If so, writing only code demanded by failing tests also has social implications:

- If the defect density can be reduced enough, QA can shift from reactive to proactive work
- If the number of nasty surprises can be reduced enough, project managers can estimate accurately enough to involve real customers in daily development
- If the topics of technical conversations can be made clear enough, programmers can work in minute-by-minute collaboration instead of daily or weekly collaboration
- Again, if the defect density can be reduced enough, we can have shippable software with new functionality every day, leading to new business relationships with customers

So, the concept is simple, but what's my motivation? Why would a programmer take on the additional work of writing automated tests? Why would a programmer work in tiny little steps when their mind is capable of great soaring swoops of design? Fear.



You are seeing the dreaded red bar. Our testing framework (JUnit, in this case) has run the little snippet of code we started with, and noticed that although we expected “10” as a result, we saw “0”. Sadness.

No, no. Failure is progress. Now we have a concrete measure of failure. That’s better than just vaguely knowing we are failing. Our programming problem has been transformed from “give me multi-currency” to “make this test work, and then make the rest of the tests work.” Much simpler. Much smaller scope for fear. We can make this test work.

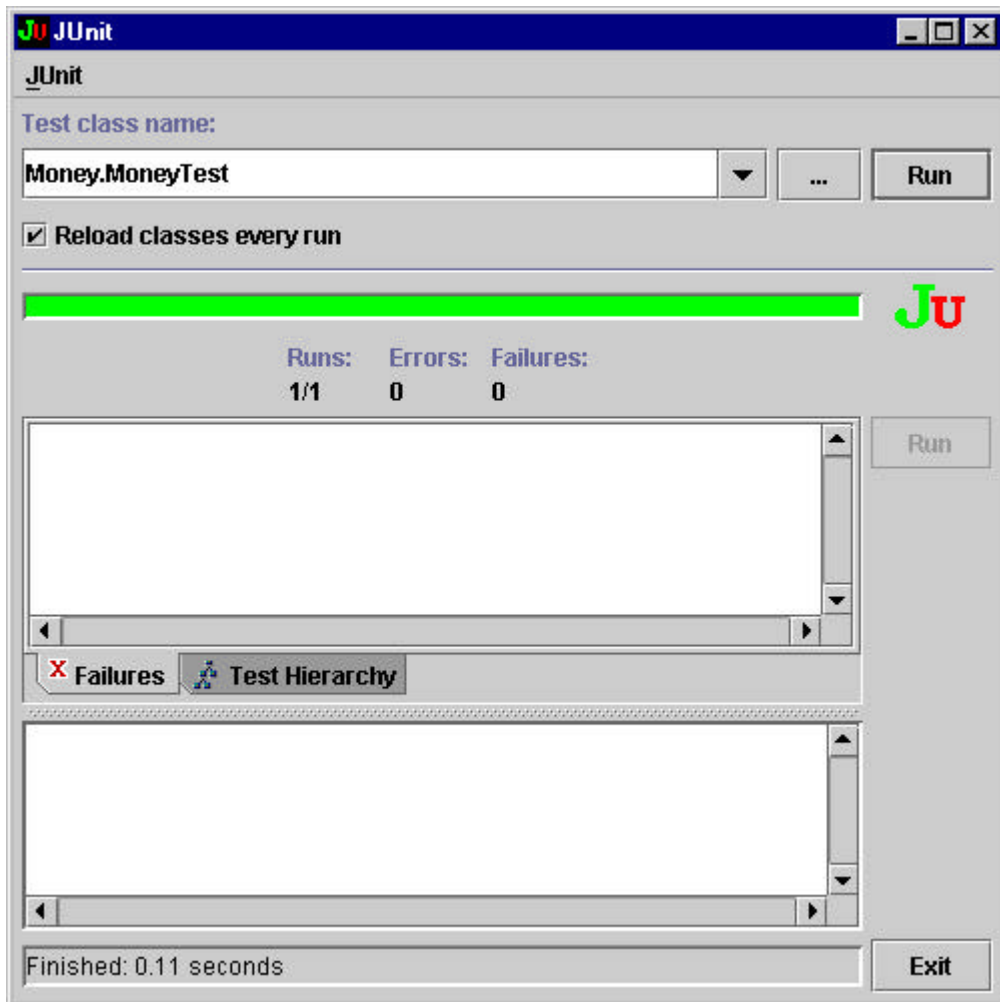
You probably aren’t going to like the solution, but the goal right now is not to get the perfect answer, the goal is to pass the test. We’ll make our sacrifice at the altar of truth and beauty later.

Here’s the smallest change I could imagine that would cause our test to pass:

Dollar

int amount= 10;

Now we get the green bar, fabled in song and story.



Oh joy, oh rapture! Not so fast, hacker boy (or girl). The cycle isn't complete. There are very few inputs in the world that will cause such a limited, such a smelly, such a naïve implementation to pass. We need to generalize before we move on. Remember, the cycle is:

1. Add a little test
2. Run all tests and fail
3. Make a little change
4. Run the tests and succeed
5. Refactor to remove duplication

Sidebar: Dependency and Duplication

Steve Freeman pointed out that the problem with the test and code as it sits is not duplication (which I have not yet pointed out to you, but I promise to as soon as this digression is over.) The problem is the dependency between the code and the test—you can't change one without changing the other. Our goal is to be able to write another test that “makes sense” to us, without having to change the code, something that is not possible with the current implementation.

To do:
\$5 + 10 CHF = \$10 if CHF:USD is 2:1
~~\$5 * 2 = \$10~~
Make "amount" private
~~Dollar side-effects?~~
Money rounding?

These are two of the three strategies for quickly getting to green:

- Fake It—return a constant and gradually replace constants with variables until you have the real code
- Obvious Implementation—type in the real implementation

When I use TDD in practice, I commonly shift between these two modes of implementation. When everything is going smoothly and I know what to type, I put in obvious implementation after obvious implementation (running the tests all the time to ensure that what's obvious to me is still obvious to the computer). As soon as I get an unexpected red bar, I back up, shift to faking implementations, and refactor to the right code. When my confidence is back, I go back to obvious implementations.

There is a third style of driving development, triangulation, which we will demonstrate in the next chapter. However, to review, we:

- Translated a design objection (side effects) into a test case that failed because of the objection
- Got the code to compile quickly with a stub implementation
- Made the test work by typing in what seemed like the right code

The translation of a feeling (disgust at side effects) into a test (multiply the same Dollar twice) is a common theme of TDD. The longer I do this, the better able I am to translate my aesthetic judgements into tests. When I can do this, my design discussions become much more interesting. First we can talk about whether the system should work like *this* or like *that*. Once we decide on the correct behavior, we can talk about the best way of achieving that behavior. We can speculate about truth and beauty all we want over beers, but while we are programming we can leave airy-fairy discussions behind and talk cases.

Money Retrospective

Let's take a look back at the Money example, both the process we used and the results. We will look at:

- Metaphor—the dramatic affect metaphor has on the structure of the design
- JUnit Usage—when we ran tests and how we used JUnit
- Code Metrics—a numerical abstract of the resulting code
- Process—we say red/green/refactor, but how much work goes into each step?
- Test Quality—how do TDD tests stack up by conventional test metrics?

Metaphor

The biggest surprise in coding this example is how different it came out this time. I have programmed Money in production at least three times that I can think of. I have used it as an example in print another half dozen times. I have programmed it live on stage (relax, it's not as exciting as it sounds...) another fifteen times. I coded another three or four times preparing for writing (I ripped out Section I and rewrote it based on early reviews.) Then, while I was writing this, I thought of using Expression as the metaphor and the design went in a completely different direction than it has gone before.

I really didn't expect the metaphor to be so powerful. A metaphor should just be a source of names, shouldn't it? Apparently not.

The metaphor Ward used for “several monies together with potentially different currencies” was a vector, like a mathematic vector where the coefficients were currencies instead of x^2 . I used MoneySum for a while, then MoneyBag (which is nice and physical), and finally Wallet (which is commoner in most folks' experience). All of these metaphors imply that the collection of Money's is flat. For example, “2 USD + 5 CHF + 3 USD” would result in “5 USD + 5 CHF”. Two values with the same currency would be merged.

The Expression metaphor freed me from a bunch of nasty issues about merging duplicated currencies. The code came out cleaner and clearer than I've ever seen it before. I'm concerned about the performance of Expressions, but I'm happy to wait until I see some usage statistics before I start optimizing.

What if I got to rewrite everything I ever wrote 20 times? Would I keep finding insight and surprise every time? Is there some way to be more mindful as I program so I can squeeze all the insight out of the first three times? The first time?

2. The number of lines of test code can be reduced by extracting common fixtures. The rough correspondance between lines of model code and lines of test code will remain, however.
3. Cyclomatic complexity is a measure of conventional flow complexity. Test complexity is 1 because there are no branches or loops in test code. Functional code complexity is low because of the heavy use of polymorphism as a substitute for explicit control flow.
4. This includes the function header and trailing brace.
5. Lines/function in the tests is inflated because we have not factored out common fixture-building code, as explained in the section on JUnit.

Process

The TDD cycle is:

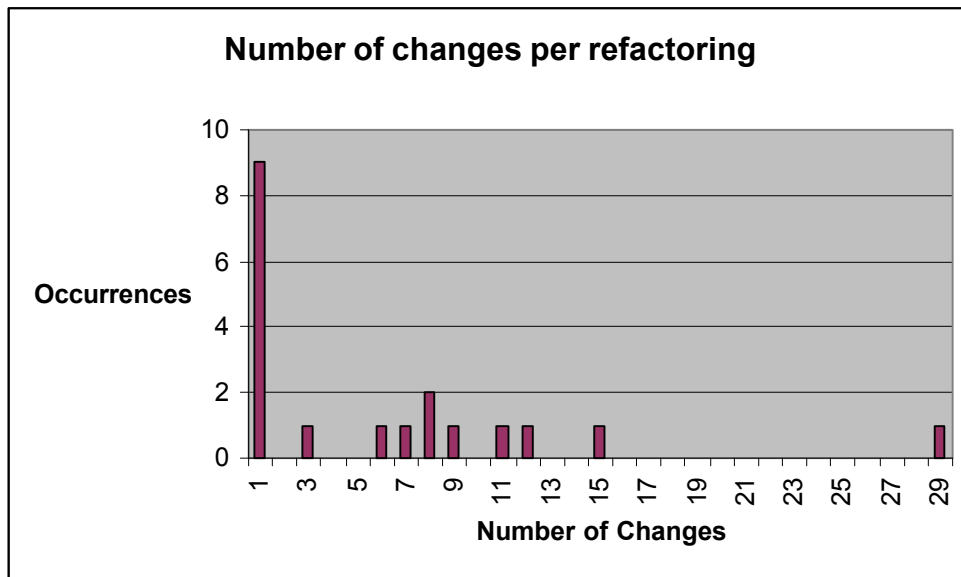
1. Write a test
2. Make it compile
3. Make it run
4. Remove duplication

Assuming that writing a test is a single step, how many changes does it take to compile, run, and refactor? (By change, I mean changing a method or class definition.)

Here is the raw data: (*Tufte, where are you when I need you?*)

Compile	Run	Refactor
4	1	7
1	2	0
0	1	0
0	1	0
1	0	8
0	0	6
0	1	11
3	2	28
0	1	7
1	1	0
5	1	0
5	1	0
0	1	2
0	1	5

0	1	14
0	1	0
0	1	10
0	1	0
0	1	0



I expect that if we gathered data for a large project, the number of changes to compile and run would remain fairly small (they could be even smaller if the programming environment understood what the tests were trying to tell it—creating stubs automatically, for instance). However, (here’s at least a master’s thesis) the number of changes per refactoring should follow a “fat tail” or leptokurtotic profile, which is like a bell curve but with more large changes. Many measurements in nature follow this profile, like price changes in the stock market.¹

Test Quality

You professional testers out there please give me some hints. What are tests that I missed that won’t run the first time?

Measure Coverage

Try defect insertion—Jester?

¹ *Fractals and Scaling in Finance*, Benoit Mandelbrot, editor, Springer-Verlag, 1997, ISBN 0387983635

Section III: Patterns

What follows are the “greatest hits” patterns for TDD. Some of the patterns are TDD tricks, some are design patterns, and some are refactorings.

The goal in these patterns is not to be comprehensive. If you want to understand testing, design patterns, or refactoring you will have to go elsewhere for mastery. If you are not familiar with these topics, there is enough here to get you going. If you are familiar with one of these topics, the patterns here will show you how the topics play with TDD.

Patterns for Test-Driven Development

Test *n*.

How do you test your software? Write an automated test.

No programmers release even the tiniest change without testing, except the very confident and the very sloppy. I'll assume that if you've gotten this far, you're neither. While you may test your changes, testing changes is not the same as *having* tests. Why does a test that runs automatically feel different than poking a few buttons and looking at a few answers on the screen?

(What follows is an influence diagram, *a la* Gerry Weinberg's Quality Software Management. An arrow between nodes means an increase in the first node implies an increase in the second. An arrow with a circle means an increase in the first node implies a decrease in the second.)

What happens when the stress level rises?

Figure 1 has Stress negatively connected to Testing negatively connected to Errors positively connected to Stress.

This is a positive feedback loop. The more stress you feel, the less testing you will do. The less testing you do, the more errors you will make. The more errors you make, the more stress you feel. Rinse and repeat.

How do you get out of such a loop? Either introduce a new element, replace one of the elements, or change the arrows. In this case we'll replace "testing" with "automated testing".

Figure 2 has Stress positively connected to Automated Testing negatively connected to Errors and Stress, and Errors positively connected to Stress.

"Did I just break something else with that change?" When I have automated tests, when I start to feel stress I run the tests. "No, the tests are all still green." The more stress I feel, the more I run the tests. Running the tests immediately gives me a good feeling, and reduces the number of errors I make, which further reduces the stress I feel.

"We don't have time to run the tests. Just release it!" The second picture isn't guaranteed. If the stress level rises high enough, it breaks down. However, with the automated tests you have a chance to choose your level of fear.

Isolated Test

How should the running of tests affect each other? Not at all.

When I was a young programmer, long long ago when we had to dig our own bits out of the snow and carry heavy buckets of them bare-footed back to our cubicles leaving bloody little footprints for the wolves to follow... Sorry, just reminiscing. My first experience of automated tests was having a set of GUI-based tests (you know, record

the keystrokes and mouse events and play them back) for a debugger I was working on (hi Jothy, hi John!). Every morning when I came in there would be a neat stack of paper on my chair describing last night's test runs (hi Al!). On good days there would be a single sheet summarizing that nothing broke. On bad days there would be many many sheets, one for each broken test. I began to dread days when I saw a pile of paper on my chair.

I took two lessons from this experience. First, make the tests so fast to run that I can run them myself, and run them often. That way I can catch errors before anyone else sees them, and I don't have to dread coming in in the morning. Second, I noticed after a while that a huge stack of paper didn't usually mean a huge list of problems. More often it meant that one test had broken early, leaving the system in an unpredictable state for the next test.

We tried to get around this problem by starting and stopping the system between each test, but it took too long, which taught me another lesson about seeking tests at a smaller scale than the whole application. But the main lesson I took was that tests should be able to ignore each other completely. If I had one test broken, I wanted one problem. If I had two tests broken, I wanted two problems.

One convenient implication of isolated tests is that the tests are order independent. If I want to grab a subset of tests and run them, I can do so without worrying that a test will break now because a prerequisite test is gone.

Performance is the usual reason cited for having tests share data. A second implication of isolated tests is that you have to work, sometimes work hard, to break your problem into little orthogonal dimensions, so setting up the environment for each test is easy and quick. Isolating tests encourages you to compose solutions out of many highly cohesive, loosely coupled objects. I always heard this was a good idea, and I was happy when I achieved it, but I never knew exactly how to regularly achieve high cohesion and loose coupling until I started writing isolated tests.

Test List

What should you test? Before you begin, write a list of all the tests you know you will have to write.

The first part of our strategy for dealing with programming stress is to never take a step forward unless we know where our foot is going to land. When we sit down to a programming session, what is it we intend to accomplish?

One strategy for keeping track of what we're trying to accomplish is to hold it all in our heads. I tried this for several years, and found I got into a positive feedback loop. The more experience I accumulated, the more things I knew that might need to be done. The more things I knew might need to be done, the less attention I had for what I was doing. The less attention I had for what I was doing, the less I accomplished. The less I accomplished, the more things I knew that needed to be done.

Just ignoring random items on the list and programming at whim did not appear to work to break this cycle.

I got in the habit of writing down everything I wanted to accomplish over the next few hours on a slip of paper next to my computer. I had a similar list, but with weekly or monthly scope pinned on the wall. As soon as I had all that written down, I knew I wasn't going to forget something. When a new item came up, I would quickly and consciously decide whether it belonged on the "now" list, the "later" list, or it didn't really need to be done at all.

Applied to test-driven development, what we put on the list are the tests we want to implement. If you want a comprehensive treatment of this subject, I recommend Bob Binder's "???". The material here is just enough to get you started. First, put on the list examples of every operation that you know you need to implement. Next, for those operations that don't already exist, put the null version of that operation. Finally, list all the refactorings that you think you will have to do to have clean code at the end of this session.

Instead of outlining the tests, we could just go ahead and implement them. There are a couple of reasons this hasn't worked for me. First, every test you implement is a bit of inertia when you have to refactor. With automated refactoring tools this is less of a problem, but when you've implemented ten tests and *then* you discover the arguments need to be in the opposite order, you are just that much less likely to go clean up. Second, if you have ten tests broken, you are a long way from the green bar. If you want to get to green quickly, you have to throw all ten tests away. If you want to get all the tests working, you are going to be staring at a red bar for a long time.

Conservative mountain climbers have a rule that you have to have three out of your four hands and feet attached at any one time. Dynamic moves where you let go of two at once are much more dangerous. The extreme form of TDD, where you are never more than one change away from a green bar, is like that three out of four rule.

As you make the tests run, the implementation will imply new tests. Write the new tests down on the list. Likewise with refactorings. "This is getting ugly." "<sigh> Put it on the list. We'll get to it before we check in."

Items that are left on the list when the session is done need to be taken care of. If you are really half way through a piece of functionality, use the same list later. If you have discovered larger refactorings that are out of scope for the moment, move them to the "later" list. I can't recall ever moving a test case to the "later" list. If I can think of a test that might not work, nothing is more important than getting it to work.

Test-First

When should you write your tests? Before you write the code that is to be tested.

You won't test after. Your goal as a programmer is running functionality.

You need a way to think about design

You need a method for scope control

Let's look at the usual influence diagram relating stress and testing (but not stress testing, that's different):

Stress above negatively connected to testing below negatively connected to stress.

Implementation Strategies

TDD is not about blindly following a set of rules for how to program. It is about intelligently choosing the size of your programming steps and the amount of feedback depending on conditions. The patterns in this section are all ways of taking small steps forwards.

Even though I'm in an sharing, caring, "can't-we-all-just-get-along" mood, it is only fair to tell you that programmers practicing TDD consistently report that they take smaller and smaller steps over time. Your brain is likely to suffer the same rot if you continue. Don't say I didn't warn you.

Fake It ('Til You Make It)

What is your first implementation once you have a broken test? Return a constant. Once you have the test running, gradually transform the constant into an expression using variables.

A simple example occurred in our implementation of xUnit.

```
return "1 run, 0 failed"
```

became:

```
return "%d run, 0 failed" % self.runCount
```

became:

```
return "%d run, %d failed" % (self.runCount , self.failureCount)
```

Fake It is a bit like driving a pylon above your head when you are climbing a rock. You haven't really gotten there yet (the test is there but the code structure is wrong). However, when you do get there, you know you will be safe (the test will still run).

There are a couple of effects that make Fake It Til You Make It powerful:

- Psychological—Having a green bar is completely different than not having a green bar. When the bar is green, you know where you stand. Refactoring from there you can do with confidence.
- Scope control—Programmers are good at imagining all sorts of future problems. Starting with one concrete example and generalizing from there prevents you from prematurely confusing yourself with extraneous concerns. You can do a better job of solving the immediate problem because you are focused. When you go to implement the next test case, you can focus on that one, too, knowing that the previous test is guaranteed to work.

Does Fake It violate the rule that says you don't write any code that isn't needed? I don't think so, because in the refactoring step you are eliminating duplication of data between the test case and the code. When I write²:

² Thanks to Dierk König for the example.

```
public void testSum() {  
    assertEquals(4, plus(2, 2));  
    assertEquals(7, plus(3,4));  
}
```

(We were careful to give the two parameters different values in the second example, so we couldn't self-righteously claim we only needed one parameter.) When we have the second example, we can abstract the implementation of plus():

```
private int plus(int augend, int addend) {  
    return augend + addend;  
}
```

Triangulation is attractive because the rules for it seem so clear. The rules for Fake It, where we are relying on our sense of duplication between the test case and the piggy implementation to drive abstraction, seem a bit vague and subject to interpretation. While they seem simple, the rules for triangulation create an infinite loop. Once we have the two assertions and we have abstracted the correct implementation for plus, we can delete one of the assertions on the grounds that it is completely redundant with the other. If we do that, however, we can simplify the implementation of plus() to just return a constant, which requires us to add an assertion.

I only use triangulation when I'm really, really unsure about the correct abstraction for the calculation. Otherwise I rely on either Obvious Implementation or Fake It.

Obvious Implementation

How do you implement simple operations? Just implement them.

Fake It and Triangulation are teensy-weensy tiny steps. Sometimes you are sure you know how to implement an operation. Go ahead.

For example, would I really use Fake It to implement something as simple as plus()? Not usually. I would just type in the obvious implementation. If I noticed I was getting surprised by red bars, I would go to smaller steps.

Keep track of how often you get surprised by red bars using Obvious Implementation. You want to maintain that red/green/refactor rhythm. Obvious Implementation is second gear. Be prepared to downshift if your brain starts writing checks your fingers can't cash.

One to Many

How do you implement an operation that works with collections of objects? Implement it without the collections first, then make it work with collections.

For example, suppose we are writing a function to sum an array of numbers. We can start with one:

```
public void testSum() {  
    assertEquals(12, sum(new int[] {5, 7}));  
}
```

Process

These patterns are about when you write a test, where you write tests, and when you stop.

One Step Test

Which test should you pick next from the list? Pick a test that will teach you something and that you are confident you can implement.

Each test should represent one step towards your overall goal. If we are looking at the following Test List, which test should we pick next?

- Plus
- Minus
- ~~Times~~
- Divide
- ~~Plus like~~
- ~~Equals~~
- ~~Equals null~~
- ~~Null exchange~~
- Exchange one currency
- Exchange two currencies
- Cross rate

There is no right answer. What is one step for me, never having implemented these objects before, will be one tenth of a step to you, with your vast experience.

When I look at a Test List, I think, “That’s obvious, that’s obvious, I have no idea, obvious, what was I thinking about with that one, ah, this one I can do.” That’s the test I implement next.

If you don’t find any test on the list that represents one step, add some new tests that would represent progress towards the items there.

A program grown from tests like this can appear to be written top-down, because you can begin with a test that represents a simple case of the entire computation. A program grown from tests can also appear to be written bottom-up, because you start with small pieces and aggregate them larger and larger.

Neither top-down nor bottom-up really describes the process helpfully. First, a vertical metaphor is a simplistic visualization of how programs change over time. “Growth” implies a kind of self-similar feedback loop where the environment affects the program and the program affects the environment. Second, if we have to have a direction in our metaphor, “known-to-unknown” is a helpful description. Known-to-unknown implies that we have some knowledge and experience on which to draw, and that we expect to learn in the course of development. Put these two together and we have programs growing from known to unknown.

Starter Test

Which test should you start with? Start by testing a variant of an operation that doesn't do anything.

The first question you have to ask with a new operation is "Where does it belong?" Until you've answered this question, you don't know what to type for the test. In the spirit of solving one problem at a time, how can we answer just this question and no other?

If you write a “realistic” test first, you will find yourself solving a bunch of problems at once:

- Where does the operation belong?
- What are the correct inputs?
- What is the correct output given those inputs?

If you begin with Answering these question a realistic will leave you too long without feedback. Red/green/refactor, red/green/refactor. You want that loop to be minutes.

You can shorten the loop by choosing inputs and outputs that are trivially easy to discover. For example, a poster on the Extreme Programming newsgroup asked about how to write a polygon reducer test-first. The input is a mesh of polygons and the output is a mesh of polygons that describes precisely the same surface, but with fewer polygons. “How can I test-drive this problem since getting a test to work requires reading Ph.D. theses?”

Starter Test provides an answer:

- The output should be the same as the input. Some configurations of polygons are already normalized, incapable of further reduction.
- The input should be as small as possible, like a single polygon, or even an empty list of polygons.

Bing! First test is running. Now for all the rest of the tests on the list.

One Step Test applies: Pick a Starter Test that will teach you something but that you are certain you can get working quickly. If you are implementing something for the Nth time, pick a test that will require an operation or two. You will be justifiably confident you can get it working. If you are implementing something hairy and complicated for the first time, you need a little courage pill immediately.

My solution is to use cheap, ugly folding tables for my computers, but buy the best chairs I can find. I have plenty of desk space, and I can easily get more, and I am fresh and ready for programming in the afternoon and the morning.

Get comfortable when you're pair programming. Clean off the desk surface enough that you can slide the keyboard back and forth. Each partner should be able to sit comfortably directly in front of the keyboard when they are driving. One of my favorite coaching tricks is to come up behind a pair that is hacking away and gently slide the keyboard so it is comfortably placed for the person typing.

Manfred Lange points out that careful resource allocation also applies to computer hardware. Get cheap/slow/old machines for individual email and surfing, and the hottest possible machines for shared development.

Testing Techniques

Child Test

How do you get a test case running that turns out to be too big? Write a smaller test case that represents the broken part of the bigger test case. Get the smaller test case running. Reintroduce the larger test case.

The red/green/refactor rhythm is so important for continuous success that when you are at risk of losing it, it is worth extra effort to maintain it. This commonly happens to me when I write a test that accidentally requires several changes to make work. Even ten minutes with a red bar gives me the willies.

When I write a test that is too big, I first try to learn the lesson. Why was it too big? What could I have done differently that would have made it smaller? How am I feeling right now.

Metaphysical navel gazing accomplished, I delete the offending test and start over. "Well, getting these three things working at once was too much. If I had A, B, and C working, though, getting the whole thing working would be a cinch."

Can I tell you a secret? Sometimes I don't even bother to delete the offending test. Shhhhhh... Do as I say, not as I do. I live with two, count 'em two, broken tests for a matter of a couple of minutes while I get the child test working. I could be making a mistake when I do this. Two broken tests could easily be a holdover from my bad old test-last-if-ever days.

Try it both ways yourself. See if you feel different, program different, when you have two tests broken. Respond as appropriate.

Mock Object

How do you test an object that relies on an expensive or complicated resource? Create a fake version of the resource that answers constants.

There is at least a book's worth of material in Mock Object, but this will serve as an introduction.

The classic example is a database. Databases take a long time to start, they are difficult to keep clean, and if they are located on a remote server, they tie your tests to a physical location on a network. The database is also a fertile source of error in development.

The solution is not to use a real database most of the time. Most tests are written in terms of an object that acts like a database, but is really just sitting in memory.

```
public void testOrderLookup() {  
    Database db= new MockDatabase();  
    db.expectQuery("select order_no from Order where cust_no is 123");  
    db.returnResult(new String[] {"Order 2" ,"Order 3"});  
    ...  
}
```

If the MockDatabase does not get the query it expects, it throws an exception. If the query is correct, it returns something that looks like a result set constructed from the constant strings.

Another value of mocks, aside from performance and reliability, is readability. You can read the test above from one end to another. If you have a test database full of realistic data, when you see that a query should have resulted in 14 replies, you have no idea why 14 is the right answer.

One of the costs of using mocks is you can't easily store expensive resources in global variables (even if they masquerade as singletons). In the case of the database example, you need to pass the database as a parameter wherever it will be used.

There have been times when I was furious at this restriction. Massimo Arnoldi and I were working on some code relying on a set of exchange rates stored in a global variable. Each test needed different subsets of the data, and sometimes they needed different exchange rates. After a while of trying to get the global variable to work, we decided one morning (courageous design decisions come more often in the morning for me) to just pass the Exchange around wherever we needed it. We thought we would have to modify hundreds of methods. In the end, we added a parameter to ten or fifteen methods, and cleaned up other aspects of the design along the way.

Mocks will encourage you down the path of carefully considering the visibility of every object, reducing the coupling in your designs.

Mock Objects add a risk to the project—what if the mock doesn't behave like the real object? You can reduce this strategy by having a set of tests for the mock that can also be applied to the real object when it becomes available.

Self Shunt

How do you test that one object communicates correctly with another? Have the object under test communicate with the test case instead of with the object it expects.

Suppose we wanted to dynamically update the green bar on the testing user interface. If we could connect an object to the TestResult, it could be notified when a test ran, when it failed, when a whole suite started and finished, and so on.

test that I would probably just go and implement if I wasn't busy writing a book.) Because this isn't supported, yet, each package should declare a class AllTests that implements a static method suite() that returns a TestSuite. Here is AllTests for the Money example:

```
public class AllTests {
    public static void main(String[] args) {
        junit.swingui.TestRunner.run(AllTests.class);
    }

    public static Test suite() {
        TestSuite result = new TestSuite("TFD tests");
        result.addTestSuite(MoneyTest.class);
        result.addTestSuite(ExchangeTest.class);
        result.addTestSuite(IdentityRateTest.class);
        return result;
    }
}
```

As you see, can give it a main() method so the class can be run directly from the IDE.

Design Patterns

Null Object

How do you represent special cases using objects? Create an object representing the special case. Give it the same protocol as the regular objects.

```
java.io.File
public boolean setReadOnly() {
    SecurityManager security = System.getSecurityManager();
    if (security != null) {
        security.checkWrite(path);
    }
    return fs.setReadOnly(this);
}
```

There are 18 places where the same “security != null” check takes place in java.io.File. While I appreciate their diligence in making files safe for the world, I'm also a bit nervous. Are they careful to always check for a null as the result of getSecurityManager()?

The alternative is to create a new class, LaxSecurity, which doesn't throw exceptions ever.

```
java.io.LaxSecurity
public void checkWrite(String path) {
}
```

If someone asks for a SecurityManager and there isn't one available, we send back a LaxSecurity instead:

```
java.lang.SecurityManager
    public static SecurityManager getSecurityManager() {
        return security != null ? security : new LaxSecurity();
    }
```

Now we don't have to worry about someone forgetting to check for null. The original code cleans up considerably:

```
java.io.File
    public boolean setReadOnly() {
        SecurityManager security = System.getSecurityManager();
        security.checkWrite(path);
        return fs.setReadOnly(this);
    }
```

Erich Gamma and I once got in an argument at an OOPSLA tutorial about whether a Null Object was appropriate somewhere in JHotDraw. I was ahead on points when he pointed out the cost of introducing the Null Object was around 10 lines of code, for which we would get to eliminate 1 conditional. I hate those late round TKOs.

Command

Template Method

Composite

Pluggable Object

Collecting Parameter

How do you collect the results of an operation that is spread over several objects? Add a parameter to the operation in which the results will be collected.

A simple example is the `java.io.Externalizable` interface. The `writeExternal` method writes an object and all the objects it references. Since the objects all have to cooperate loosely to get written out, the method is passed a parameter, an `ObjectOutput`, as the collecting parameter:

```
java.io.Externalizable
    public interface Externalizable extends java.io.Serializable {
        void writeExternal(ObjectOutput out) throws IOException;
    }
```

Value Object

How do you design objects that will be widely shared, but for whom identity is unimportant? Set their state when they are created and never change it. Operations on the object always return a new object.

Objects are wonderful. I can say that here, can't I? Objects are a great way to organize logic for later understanding and growth. However, there is one little problem (okay, more than one, but this one will do for now.)

Suppose I (an object) have a Rectangle. I compute some value based on the Rectangle, like its area. Later, someone politely asks me for my Rectangle, and I, not wanting to appear uncooperative, give it to them. Moments later, lo and behold, the Rectangle has been changed behind my back. The area I computed earlier is out of date, and there is no way for me to know.

This is the classic aliasing problem. If two objects share a reference to a third, if one object changes the referred object, the other object better not rely on the state of the shared object.

There are several ways out of the aliasing problem. One solution is never to give out the objects that you rely on, but instead to always make copies. This can get expensive in time and space, and ignores those times when you want to share changes to a shared object. Another solution is Observer, where you explicit register with objects on which you rely and expect to be notified when they change. Observer can make control flows difficult to follow, and the logic for setting up and removing the dependencies gets ugly.

Another solution is to treat the object as less than an object. Objects have state that change over time. We can, if we choose, eliminate the "that change over time". If I have an object and I know it won't change, I can pass around references to it all I want, knowing that aliasing won't be a problem. There can be no hidden changes to a shared object if there are no changes.

I remember puzzling over Integers when I was first learning Smalltalk. If I change bit 2 to a 1, why don't all 2's become 6's?

```
a := 2.  
b := a.  
a := a bitAt: 2 put: 1.  
a => 6  
b => 2
```

Because Integers are really values masquerading as objects. In Smalltalk this is literally true of small integers, and simulated in the case of integers that don't fit in a single machine word. When I set that bit, what I get back is a new object with the bit set, not the old one with the bit changed.

When implementing a Value Object, every operation has to return a fresh object, leaving the original unchanged. Users have to be aware they are using a Value Object and store the result (as in the example above.) All of these object allocations can create performance problems, which should be handled like all performance problems,

when you have realistic data sets, realistic usage patterns, profiling data, and complaints about performance.

I have a tendency to use Value Object whenever I have a situation that looks like algebra—geometric shapes being intersected and unioned, unit values where units are carried around with a number, symbolic arithmetic. Any time Value Object makes the least sense I try it, because it makes reading and debugging so much easier.

All Value Objects have to implement equality (and in many languages by implication they have to implement hashing.) If I have this Contract and that Contract and they aren't the same object, then they are different, not equal. However, if I have this five francs and that five francs, it doesn't matter if they are the same five francs, five francs are five francs and they should be equal.

Imposter

Refactoring

There is a brief description of how to accomplish each refactoring in small steps. More importantly, each refactoring discusses why you might want to use it.

In TDD we use “refactoring” in an interesting way. Usually, a refactoring cannot change the semantics of the program under any circumstances. In TDD, the circumstances we care about are the tests that are already passing. So, for example, we can replace constants with variables in TDD can call this operation, in good conscience, a refactoring, because it doesn't change the set of tests that pass. The only circumstance under which semantics are preserved may actually be our one test case. Any other test case that was passing would fail. However, we don't have those tests yet, so we don't worry about them.

This kind of weird “refactoring with respect to tests” places a burden on you to have enough tests that as far as you know, a refactoring with respect to the tests is the same as a refactoring with respect to all possible tests, at least by the time you're done. It's no excuse to say, “I knew there was a problem, but the tests all passed so I checked the code in.” Write more tests.

Reconcile Differences

How do you unify two similar looking pieces of code? Gradually bring them closer. Unify them only when they are absolutely identical.

Refactoring can be a nerve wracking experience. The easy ones are obvious. If I extract a method, as long as I do it mechanically correctly, there is very little chance of changing the system's behavior. Some refactorings push you to examine the control flows and data values carefully. A long chain of reasoning leads you to believe that the change you are about to make won't change any answers. Those are the refactorings that enhance your hairline.

Such a leap of faith refactoring is exactly what we're trying to avoid with our strategy of small steps and concrete feedback. While you can't always avoid leapy refactorings, you can reduce their incidence.

For example, suppose we have code in two subclasses that we think does exactly the same thing:

I'm having trouble finding an example

We could just write a method in the superclass that we think implements the same algorithm. That would be a big step. However, if the two methods were exactly the same, we could put the method in the superclass without any worries.

The same general strategy works for all refactorings—if you want to make a big change, think about how the last step of the change could be trivial, then work backwards. For example, if you want to remove several subclasses, the trivial last step is if a subclass contains nothing. Then the subclass can be replaced by the superclass without changing the behavior of the system. One by one, empty out the subclasses and, when they are empty, replace references to them by references to the superclass.

Isolate Change

How do you change one part of a multi-part method or object? First, isolate the part that has to change.

If you want to be absolutely sure you aren't changing anything accidentally (not that you'll be right always), before you change it, make a place for it. We saw an example in working with Money. The conversion code said:

```
return new Note(source.amount / rate, currency)
```

We wanted to change rate lookup. First we extracted the access to the instance variable rate into its own method:

```
return new Note(source.amount / findRate(), currency)
private int findRate() {
    return rate;
}
```

Now we can work on findRate() in isolation, without having to worry about affecting the parts of convert() that have nothing to do with rate lookup.

The picture that comes to my mind is surgery, where all of the patient except the part to be operated on is draped. The draping leaves the surgeon with only a fixed set of variables. Now, we could have long arguments over whether this abstraction of a person to a lower left quadrant abdomen leads to good health care, but at the moment of surgery, I'm kind of glad the surgeon can focus.

You may find that once you've isolated the change and then made the change, that the result is so trivial that you can undo the isolation. If we found that really all we needed was to return the instance variable in findRate(), we should consider inlining findRate() everywhere it is used and deleting it. Don't make these changes automatically, however. Balance the cost of an additional method with the value of having an additional concept explicit in the code.

Some possible ways to Isolate Change are Extract Method (the most common), Extract Object, and Method Object.

Migrate Data

How do you move from one representation? Temporarily duplicate the data.

How:

Here is the internal-to-external version:

1. Add an instance variable in the new format
2. Set the new format variable everywhere you set the old format
3. Use the new format variable everywhere you use the old format
4. Delete the old format
5. Change the external interface to reflect the new format

Sometimes, though, you want to change the API first. Then you should:

1. Add a parameter in the new format
2. Translate from the new format parameter to the old format internal representation
3. Delete the old format parameter
4. Replace uses of the old format with the new format
5. Delete the old format

Why:

One to Many creates a data migration problem every time. Suppose we wanted to implement TestSuite using One to Many. We would start with:

```
def testSuite(self):
    suite = TestSuite()
    suite.add(WasRun("testMethod"))
    suite.run(self.result)
    assert("1 run, 0 failed" == self.result.summary())
```

Which is implemented (in the “One” part of One to Many) by:

```
class TestSuite:
    def add(self, test):
        self.test = test
    def run(self, result):
        self.test.run(result)
```

Now we begin duplicating data. First we initialize the collection of tests:

```
TestSuite
def __init__(self):
    self.tests = []
```

Everywhere “test” is set, we add to the collection, too:

TestSuite

```
def add(self, test):  
    self.test = test  
    self.tests.append(test)
```

Now we use the list of tests instead of the single test. For purposes of the current test cases this is a refactoring (it preserves semantics) because there is only ever one element in the collection.

TestSuite

```
def run(self, result):  
    for test in self.tests:  
        test.run(result)
```

We delete the now-unused instance variable “test”:

TestSuite

```
def add(self, test):  
    self.tests.append(test)
```

You can also use stepwise data migration when moving between equivalent formats with different protocols, as in moving from Java’s Vector/Enumerator to Collection/Iterator. *Would this make a better example, since the above example is duplicated elsewhere in the text?*

Extract Method

Inline Method

Extract Interface

Move Method

Method Object

How do you represent a complicated method that requires several parameters and local variables? Make an object out of the method.

How:

1. Create an object with the same parameters as the method.
2. Make the local variables also instance variables of the object.
3. Create one method called "run()", whose body is the same as the body of the original method.
4. In the original method, create a new object and invoke run().

Why:

Method Objects are useful in preparation for adding a whole new kind of logic to the system. For example, you might have several methods involved in computing the cash flow from component cash flows. When you want to start computing the net present value of the cash flows, you can first create a Method Object out of the first style of computation. Then you can write the new style of computation with its own, smaller-scale, tests. Then plugging in the new style will be a single step.

Method Objects are also good for simplifying code that doesn't yield to Extract Method. Sometimes you'll find a block of code that has a bunch of temporary variables and parameters, and every time you try to extract a piece of it you have to carry along five or six temps and parameters. The resulting extracted method doesn't look any better than the original code, because the method signature is so long. Creating a Method Object gives you a new namespace in which you can extract methods without having to pass anything.

Add Parameter

How do you add a parameter to a method?

How:

1. If the method is in an interface, add the parameter to the interface first
2. Use the compiler errors to tell you what other code you need to change

Why:

Adding a parameter is often an extension step. You got the first test case running without needing the parameter, but in this new circumstance you have to take more information into account in order to compute correctly.

Adding a parameter can also be part of migrating from one data representation to another. First you add the parameter, then you delete all uses of the old parameter, then you delete the old parameter.

Method Parameter to Constructor Parameter

How do you move a parameter from a method or methods to the constructor?

How:

1. Add a parameter to the constructor
2. Add an instance variable with the same name as the parameter
3. Set the variable in the constructor
4. One by one, convert references to "parameter" to "this.parameter"
5. When no more references exist to the parameter, delete the parameter from the method and all caller
6. Remove the now-superfluous "this." from references
7. Rename the variable correctly

Why:

If you pass the same parameter to several different methods in the same object, you can simplify the API by passing the parameter once (eliminating duplication). You can run this refactoring in reverse if you find that an instance variable is only used in one method.

Mastering TDD

I hope to raise questions here for you to ponder as you integrate TDD into your own practice. Some of the questions are small, and some are large.

How large should your steps be?

There are really two questions lurking here:

- How much ground should each test cover?
- How many intermediate stages should you go through as you refactor?

You could write the tests so they each encouraged the addition of a single line of logic and a handful of refactorings. You could write the tests so they each encouraged the addition of hundreds of lines of logic and hours of refactoring. Which should you do?

Part of the answer is that you should be able to do either. The tendency of TDDers over time is clear, though—smaller steps. However, folks are experimenting with driving development from application-level tests, either alone or in conjunction with the programmer-level tests we've been writing.

At first when you refactor, you should be prepared to take lots of little tiny steps. Manual refactoring is prone to error, and the more errors you make and only catch later, the less likely you are to refactor. Once you've done a refactoring 20 times by hand in little tiny steps, experiment with leaving out some of the steps.

Automated refactoring accelerates refactoring enormously. What would have taken you 20 manual steps now becomes a single menu item. An order of magnitude change in quantity generally constitute a change in quality, and this is true of automated refactoring. When you know you are supported by an excellent tool, you become much more aggressive in your refactorings, trying many more experiments to see how the code wants to be structured.

The Refactoring Browser for Smalltalk is as I write still the best refactoring tool available. Java refactoring support is appearing in many Java IDEs, and refactoring support is sure to spread quickly to other languages and environments.

How much feedback do you need?

How many tests should you write? Here's a simple problem—given three integers representing the length of the sides of a triangle, return:

- 1 if the triangle is equilateral
- 2 if the triangle is isosceles
- 3 if the triangle is scalene

and throw an exception if the triangle is not well formed.

Go ahead, try the problem (my Smalltalk solution is listed at the end of this question).

I wrote 6 tests (kind of like Name That Tune, “I can code that problem in four tests.” “Code that problem.”) Bob Binder, in his comprehensive book *Testing Object-Oriented Software*, wrote 65 for the same problem. You’ll have to decide, from experience and reflection, about how many tests you want to write.

I think about Mean Time Between Failure (MTBF) when I think about how many tests to write. For example, Smalltalk integers act like integers, not like a 32-bit counter, so it doesn’t make sense to test MAXINT. Well, there is a MAXINT, but it has to do with how much memory you have. Do I need to write a test that fills up memory with extremely large integers? How will that affect the MTBF of my program? If I’m never going to get anywhere close to that size of triangle, my program is not measurably more robust with such a test than without it.

Whether a test makes sense to write depends on how carefully you measure MTBF. If you are trying to get from an MTBF of 10 years to an MTBF of 100 years in your pacemaker, it probably makes sense, unless you can demonstrate in some other way that you never get triangles that large.

TDD’s view of testing is pragmatic. The tests are a means to an end, the end being code in which we have great confidence. If our knowledge of the implementation gives us confidence in the absence of a test, we will not write that test. Black box testing, where we deliberately choose to ignore the implementation, has some advantages. By ignoring the code, it demonstrates a different value system—the tests are valuable alone. It’s an appropriate attitude to take in some circumstances. However, it is different than TDD.

How does the programming language and environment influence TDD?

Try TDD in Smalltalk with the Refactoring Browser. Try it in C++ with vi. How does your experience differ?

In programming languages and environments where cycles are harder to come by (TDD cycles—test/compile/run/refactor), you will likely be tempted to take larger steps:

- Cover more ground with each test
- Refactor with fewer intermediate steps

Does this make you go faster or slower.

In programming languages where cycles are plentiful, you will likely be tempted to try lots more experiments. Does this help you go faster or reach better solutions, or would you be better off institutionalizing some kind of time for pure reflection (reviews or literate programs)?

How can you use TDD to teach programming, design, and/or testing?

Should you use TDD to teach CS1? Should (or how should) experienced TDDers use it to learn new languages or environments? How should TDD be modified to maintain the current pedagogical separation between programming, testing, and design? How should the current pedagogical separation between programming, testing, and design be modified to take advantage of TDD? How does TDD change when pair programming? How about if the pair are both inexperienced? If one is inexperienced and one is experienced? If both are experienced? If both are experienced but in different domains?

Can you test-drive enormous systems?

Does TDD scale to extremely large systems? What new tests would you have to write? What new kinds of refactorings would you need?

Can you drive development with application-level tests?

The problem with driving development with small scale tests (I call them “unit tests”, but they don’t match the accepted definition of unit tests very well. *Why?*) is that you run the risk of implementing what you think a user wants, but having it turn out to be not what they wanted at all. What if we wrote the tests at the level of the application? Then the users (with some help) could write tests themselves for what exactly they wanted.

There is a technical problem—fixturing. How can you write and run a test for a feature that doesn’t exist yet. There always seems to be some way out of this problem, typically by introducing an interpreter which gracefully signals an error when it comes across a test that it doesn’t know how to interpret yet.

There is also a social problem with application test driven development. Writing tests is a new responsibility for users (by which I really mean a team that includes users), and that responsibility comes at a new place in the development cycle, namely before implementation begins. Organizations resist this kind of shift of responsibility. It will require concerted effort (that is, the effort of many people on the team working in concert) to get application tests written first.

TDD as described in this book is a technique that is entirely under your control. You can pick it up and start using it today if you so choose. Mixing up the rhythm of red/green/refactor, the technical issues of application fixturing, and the organizational change issues surrounding user-written tests is unlikely to be successful. The One Step Test rule applies. Get red/green/refactor going in your own practice, then spread the message.

Another aspect of ATDD is the length of the cycle between test and feedback. If a customer wrote a test and ten days later it finally worked, you would be staring at a red bar most of the time. I think I would want to still do programmer-level TDD so:

- I got immediate green bars
- I simplified the internal design

Is TDD sensitive to initial conditions?

Some orders in which you take the tests seem to work very smoothly.

Red/green/refactor/red/green/refactor. You can take the same tests and implement them in a different order, and it seems like there is not a way to advance in small steps. Is it really true that one sequence of tests is an order of magnitude faster/easier to implement than another? Is this just because my implementation technique is not up to the challenge? Is there something about the tests that should tell me to tackle them in a certain order? If TDD is sensitive to initial conditions in the small, is it predictable in the large? (In the same way that little eddies in the Mississippi are unpredictable, but you can count on 2,000,000 cfs. more or less at the river mouth.)

Why does TDD work?

I saved the weirdest for last. Let's assume for the moment that TDD helps teams productively build loosely coupled, highly cohesive systems with low defect rates and low cost maintenance profiles. (I'm claiming no such thing in general, just for myself, but I trust you to imagine impossible things.) How could such a thing happen?

An answer comes from the fevered imagination of complex systems. The inimitable Philip says:

Adopt programming practices that "attract" correct code as a limit function, not as an absolute value. If you write [UnitTests](#) for every feature, and if you Refactor to simplify code between each step, and if you add features one at a time and only after all the [UnitTests](#) pass, you will create what mathematicians call an "iterative dynamic attractor". This is a point in a state space that all flows converge on. Code is more likely to change for the better over time instead of for the worse; the attractor approaches correctness as a limit function.

This is the "correctness" that nearly all programmers get by with (except, of course, for medical or aerospace software). But it's better to explicitly understand the attractor concept than deny it or disregard its importance.

I hope that gives you something to chew on.